AIC211 - AI for Cybersecurity

PROJECT TEMPLATE
& DETAILED GRADING RUBRIC

Document for Students and Instructors

Proposed Version Aligned with the Course Syllabus

Syllabus anchor points
The project is an on-going assessment component completed in groups under instructor supervision.
Project weight in the assessment structure: 30%.
Primary CLO assessed directly: CLO10 - design, implement, and present an end-to-end AI pipeline for a real-world cybersecurity problem.
Supporting capabilities often demonstrated in the project: CLO4, CLO5, CLO6, CLO7, CLO8, and CLO9.

PART A - PROJECT TEMPLATE FOR STUDENTS

Purpose. The project requires each student team to build an end-to-end AI for Cybersecurity solution: define the problem, collect or select data, preprocess the data, engineer features, train the model, evaluate performance, analyze risks, and present the operational value of the solution.

Item	Recommended requirement
Format	Team-based work under instructor supervision; submission includes a report and a presentation/demo.
Expected outcome	A complete AI pipeline for a cybersecurity problem, including data, a model, evaluation, and clear operational conclusions.
Primary CLO	CLO10 (directly), while also demonstrating capabilities from CLO4–CLO9 depending on the topic.
Minimum outputs	01 proposal, 01 runnable source package, 01 final report, 01 presentation slide deck, and 01 demo or experimental walkthrough.
Academic integrity principle	No plagiarism; clearly cite datasets, libraries, pretrained models, and the extent of GenAI/LLM usage; the team must identify its own implementation work.

1. Suggested project topics

- Phishing email or spam detection using classical ML or deep learning.
- Malicious URL / phishing URL classification.
- Network intrusion detection or anomaly detection on flow/log data.
- Malware / malicious event classification from static or dynamic features.
- Account takeover / fraud / abuse detection from behavioral data.
- Log triage or alert enrichment using an LLM/RAG/agentic workflow with guardrails.
- Adversarial ML or robustness evaluation for a security model.
- Other topics approved by the instructor, provided they fit the scope of AI for Cybersecurity.

2. Conditions for project approval

1.
2.

2.1. The problem must belong to the domain of cybersecurity or digital fraud prevention, with a clear contextual description.

2.2. The data must be legally usable for academic work; if synthetic data or a public dataset is used, the source must be cited.

2.3. The project must include at least one machine learning or deep learning model; purely theoretical descriptions are not acceptable.

2.4. The project must include quantitative evaluation using appropriate metrics such as precision, recall, F1, and a confusion matrix; if the data are imbalanced, the handling approach must be stated.

2.5. The project must discuss practical deployment risks: false positives, false negatives, drift, poisoning/evasion, privacy, or prompt injection (if an LLM is used).

3. Proposal / project charter template (submitted by the team in the early stage)

Teams should complete the following template
3.1. Project title:
3.2. Team members, student IDs, and the role of each member:
3.3. Cybersecurity problem to be addressed:
3.4. Project objectives (business/security objective and technical objective):
3.5. ML problem type: classification / anomaly detection / clustering / NLP / deep learning / LLM-assisted workflow:
3.6. Planned dataset: source, size, data type, labeling method, and data risks:
3.7. Planned baseline:
3.8. Planned main model:
3.9. Evaluation plan: data split, metrics, baseline comparison, imbalance handling:
3.10. Tools and environment: Python, notebook, framework, GPU/CPU, repo:
3.11. Weekly work plan:
3.12. Key risks and mitigation plan:

3.13. Final outputs the team commits to deliver:

4. Required structure of the final project

1.
2.
3.
4.

4.1. Problem definition: describe the threat/use case, actor, input data, and desired output.

4.2. Related background: briefly summarize the relevant technical concepts and explain why this approach was selected.

4.3. Dataset and data understanding: source, schema, class distribution, and data quality issues.

4.4. Preprocessing and feature engineering: cleaning, encoding, vectorization, feature extraction, and balancing.

4.5. Modeling: baseline, main model, hyperparameters, and rationale for selection.

4.6. Evaluation: train/validation/test or cross-validation; confusion matrix; precision/recall/F1; error analysis.

4.7. Security interpretation: how false positives affect SOC/operations; what risks false negatives create.

4.8. Robustness / limitation analysis: drift, adversarial issues, label noise, privacy, and generalization.

4.9. Deployment thinking: if deployed in practice, how the pipeline would run, who would use it, and what logging/monitoring would be required.

4.10. Conclusion and future work: what the team learned, what remains incomplete, and what the next steps are.

5. Required deliverables for project submission

Deliverable	Minimum content
01 final report (.docx/.pdf)	8–15 pages of main content, clearly presenting the pipeline, results, and analysis.
01 presentation slide deck	Approximately 10–15 slides focusing on the problem, data, method, results, demo, and lessons learned.
01 source package / repo	The source code is clearly structured and includes a README, requirements, run instructions, and a description of the data directory.

01 results file	Metric tables, confusion matrix, visual figures, execution logs, or notebook output.
01 demo / short video (encouraged)	May be a notebook walkthrough, command-line demo, or UI prototype.
Declaration of AI tool usage	Clearly state where ChatGPT/Copilot/LLMs were used; the team remains fully responsible for academic integrity.

6. Suggested final report structure

- Cover page: course name, project title, team members, instructor, and semester.
- Executive summary (0.5–1 page).
  1. Introduction and problem statement.
  2. Dataset and exploratory analysis.
  3. Methodology and pipeline design.
  4. Experiments and evaluation results.
  5. Error analysis and security discussion.
  6. Limitations, ethics, and operational risks.
  7. Conclusion and future work.
- References and appendices (if any).

7. Pre-submission self-check checklist

☐ The team has clearly described the security problem and its practical value.

☐ The dataset has a clear source and a clear description of class distribution and data issues.

☐ A baseline is included for comparison, rather than presenting only one model.

☐ At minimum, the following metrics are included: precision, recall, F1, and a confusion matrix.

☐ False positives / false negatives are analyzed from a security operations perspective.

☐ If data imbalance appears, the team has described how it was handled.

☐ If LLM/GenAI is used, the team has stated the guardrails and the risks of hallucination/prompt injection.

☐ The code can be reproduced by following the README.

☐ The slides are consistent with the report and the numbers in the code/output.

☐ All figures, tables, and references have been properly cited.

PART B - DETAILED GRADING RUBRIC

Scoring principle. The rubric below scores the project on a 100-point scale, which is then converted to the course project weight. Instructors may use it directly or adjust it slightly to fit a specific topic, but the core intent should remain unchanged: evaluate technical quality, security thinking, the completeness of the pipeline, and the ability to explain the results.

Criterion	Weight
1. Problem definition and security context	10
2. Data, EDA, and data quality	15
3. Preprocessing and feature engineering	10
4. Model design and baseline	15
5. Experiments and evaluation	20
6. Security analysis, robustness, and limitations	10
7. Code completeness / reproducibility	10
8. Report, slides, presentation, and defense	10
Total	**100**

8. Detailed rubric across 4 performance levels

1. Problem definition and security context (10 points)

Performance level	Description
Excellent (9–10)	The problem is clear and within the cybersecurity domain, with a specific actor/use case/threat model and desired output; the technical objective is tightly aligned with the operational objective.
Good (7–8.5)	The problem is valid and within the course scope; it is described fairly clearly, though operational impact or threat context is not explored deeply.
Satisfactory (5–6.5)	The problem is stated but remains generic; the security context is still vague; it is not clearly shown why AI/ML is appropriate.

Unsatisfactory (0–4.5)	The problem is vague, out of scope, or shows little to no security context.

2. Data, EDA, and data quality (15 points)

Performance level	Description
Excellent (13.5–15)	The data source is transparent; the schema, class distribution, missing values, label noise, or bias are described; EDA includes clear visualizations and insights.
Good (10.5–13)	The data are described and basic EDA is provided; some data issues are identified, though not in depth.
Satisfactory (7.5–10)	The data are described only superficially; EDA is limited; data quality is barely discussed.
Unsatisfactory (0–7)	The data source is unclear or the data handling is superficial and lacks evidence.

3. Preprocessing and feature engineering (10 points)

Performance level	Description
Excellent (9–10)	The preprocessing pipeline is appropriate, with a clear explanation of encoding/vectorization/feature extraction; if the data are imbalanced, the handling is appropriate.
Good (7–8.5)	Preprocessing and feature engineering are in the right direction, with a reasonably clear explanation.
Satisfactory (5–6.5)	Preprocessing is included, but the rationale is missing or inconsistent with the data type.
Unsatisfactory (0–4.5)	The preprocessing is incorrect or almost nonexistent.

4. Model design and baseline (15 points)

Performance level	Description
Excellent (13.5–15)	A clear baseline is provided; the main model is chosen with justification; the experiment design is appropriate; tuning or comparison is reasonable.
Good (10.5–13)	A baseline and a main model are included; the model choice is generally reasonable but not deeply justified.
Satisfactory (7.5–10)	Model training is present, but either the baseline is missing or the rationale for model selection is weak.
Unsatisfactory (0–7)	The baseline is missing, the model is chosen arbitrarily, or the implementation has serious flaws.

5. Experiments and evaluation (20 points)

Performance level	Description
Excellent (18–20)	The evaluation is rigorous; the data split or cross-validation is appropriate; the confusion matrix, precision, recall, and F1 are reported; error analysis is included; conclusions are data-driven.
Good (14–17.5)	The main metrics and comparisons are reasonably clear; error analysis is moderate.
Satisfactory (10–13.5)	Metrics are included, but some are missing or the interpretation remains superficial.
Unsatisfactory (0–9.5)	The evaluation is weak, key metrics are missing, or the conclusions do not match the evidence.

6. Security analysis, robustness, and limitations (10 points)

Performance level	Description
Excellent (9–10)	False positives / false negatives are analyzed clearly in the security context; drift, adversarial risk, privacy, prompt injection, or deployment risk are identified where relevant.
Good (7–8.5)	Limitations and operational risks are discussed, but not yet in a systematic way.
Satisfactory (5–6.5)	Limitations are mentioned only in general terms, with weak linkage to security practice.
Unsatisfactory (0–4.5)	There is almost no discussion of risk or limitations.

7. Code completeness / reproducibility (10 points)

Performance level	Description
Excellent (9–10)	The code is clean, structured, and runnable, with a clear README; results are reproducible; the directory is organized and dependencies are managed.
Good (7–8.5)	Most of the code runs successfully; the usage documentation is reasonably clear.
Satisfactory (5–6.5)	The code is somewhat messy or difficult to rerun, but it still shows genuine implementation effort.
Unsatisfactory (0–4.5)	The code has major errors, is difficult to verify, or is insufficient for reproduction.

8. Report, slides, presentation, and defense (10 points)

Performance level	Description
Excellent (9–10)	The report is coherent and well presented, the slides are concise, the demo/presentation is clear, and the defense is confident.
Good (7–8.5)	The presentation is good, the logic is fairly clear, and the defense is reasonably solid.
Satisfactory (5–6.5)	The content is present, but the delivery is fragmented or the team lacks confidence during the defense.
Unsatisfactory (0–4.5)	The report/slides are weak, the presentation is underprepared, and the results are difficult to defend.

9. Suggested bonuses and penalties

Case	Suggested adjustment
A working demo is provided, with a dashboard/UI or a near-production pipeline	+1 to +3 bonus points within the project score

A reasonable comparison is provided across classical ML, DL, and/or an LLM workflow	+1 to +2 points
AI tool usage is not declared, or source code/materials are copied	Severe deduction under academic integrity rules; the project may receive 0 points
Individual member contributions cannot be demonstrated	The instructor may adjust individual scores
The code cannot be run, or the reported figures do not match the evidence	Deduct 2 to 10 points depending on severity

10. Quick scoring sheet for instructors

Criterion	Maximum score	Awarded score	Short comments
1. Problem definition and security context	10		
2. Data, EDA, and data quality	15		
3. Preprocessing and feature engineering	10		
4. Model and baseline	15		
5. Experiments and evaluation	20		
6. Security discussion / robustness / limitations	10		
7. Code and reproducibility	10		
8. Report, presentation, defense	10		
Total	100		

11. Suggested individual score adjustment within a group

- Default: the same group score for all members if contributions are relatively balanced.
- If work logs or peer evaluation show clear differences in contribution, the instructor may apply an individual multiplier (for example 0.8–1.1).
- The team should submit a task allocation sheet and evidence of contribution: repo commits, notebook ownership, slide/report section ownership, and demo preparation.

12. Notes for use

This document is an operational course template. Instructors may customize submission milestones, group size, the topic list, and bonuses/penalties to fit the actual course schedule, as long as the course learning outcomes are not distorted.

Basis for development: CLO10 requires students to “design, implement, and present an end-to-end AI pipeline addressing a real-world cybersecurity problem”; the project is a 30% on-going assessment component, supervised by the instructor, and completed in groups.